

Supplementary Material A:

End-to-End Workflow and Usage Tutorial for OppNDA

Contents

1	Introduction	2
1.1	Workflow Overview	2
1.2	Prerequisites	2
2	Installation and Setup	3
2.1	Method 1: Automated Setup Scripts	3
2.1.1	Windows	3
2.1.2	Linux/macOS	3
2.2	Method 2: Manual Installation	4
2.3	Method 3: Docker Installation	4
2.3.1	Using Docker Compose (Recommended)	4
2.3.2	Manual Docker Build	4
2.4	Accessing the Web Interface	4
3	Scenario Configuration	4
3.1	Importing Existing Configuration Files	6
3.2	Scenario Settings	6
3.3	Network Interfaces	6
3.4	Host Groups	7
3.5	Message Events	7
3.6	Report Selection	9
3.7	Saving Configuration	9
4	Running the ONE Simulator	9
4.1	Starting a Simulation	9
4.2	Monitoring Progress	10
4.3	Terminating a Simulation	10
5	Post-Processing: Report Averaging	10
5.1	Understanding Filename Patterns	11
5.2	Configuring the Average class	11
5.3	Pattern Builder	11
5.4	Running the 'Averager'	12
5.5	Understanding Averaged Output	12
6	Post-Processing: Visualization and Analysis	13
6.1	Configuring Analysis	13
6.2	Available Plot Types	13
6.3	Plot Settings	13
6.4	Running Analysis	14
6.5	Viewing Generated Plots	14
7	Regression Analysis	14
7.1	Preparing Input Data	14
7.2	Configuring Regression	16
7.3	Running Regression	16

1 Introduction

OppNDA (Opportunistic Network Data Analyzer) is a comprehensive framework that streamlines the workflow for configuring, running, and analyzing simulations in the ONE simulator. This document provides a step-by-step walkthrough of the framework, supplementing the main manuscript titled **OppNDA: A Modular and Scalable Automation Framework for Streamlining DTN Research with the ONE simulator**, submitted to *Simulation Modelling Practice and Theory*.

This supplementary material is intended for reviewers and researchers interested in reproducing or extending the OppNDA analysis workflow. It emphasizes practical usage rather than implementation details, which are provided separately in Supplementary Material B.

1.1 Workflow Overview

The complete OppNDA workflow consists of the following stages:

1. **Setup:** Install dependencies and launch the application
2. **Configuration:** Define simulation scenarios using the web interface
3. **Simulation:** Execute the ONE simulator with configured parameters
4. **Averaging:** Aggregate results across multiple simulation parameters
5. **Analysis:** Generate visualizations
6. **Regression:** Train machine learning models on simulation data

1.2 Prerequisites

Before proceeding, ensure you have the following:

- Python 3.9 or higher
- ONE simulator (for generating simulation reports)
- A modern web browser (Chrome, Firefox, Edge)
- Git (for cloning the repository)
- Docker (optional, for containerized deployment)

2 Installation and Setup

OppNDA provides multiple installation methods to accommodate different environments and preferences.

2.1 Method 1: Automated Setup Scripts

The simplest installation method uses the provided setup scripts.

2.1.1 Windows

1. Download or clone the OppNDA repository:

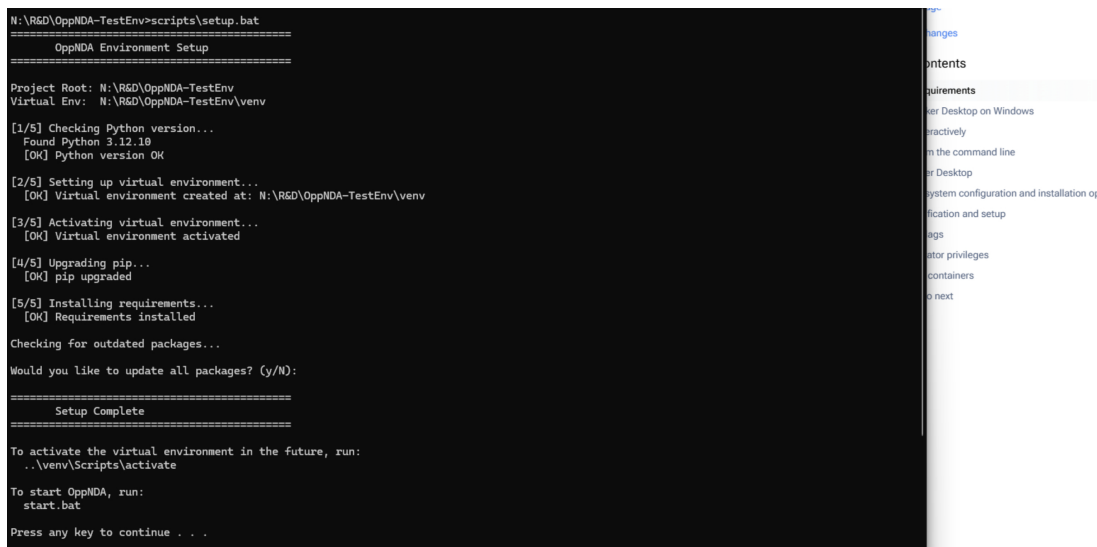
```
git clone https://github.com/**/.git
cd OppNDA
```

2. Run the setup script:

```
scripts\setup.bat
```

3. Launch the application:

```
scripts\start.bat
```



```
N:\RGD\OppNDA-TestEnv>scripts\setup.bat

=====
OppNDA Environment Setup
=====

Project Root: N:\RGD\OppNDA-TestEnv
Virtual Env: N:\RGD\OppNDA-TestEnv\venv

[1/5] Checking Python version...
Found Python 3.12.18
[OK] Python version OK

[2/5] Setting up virtual environment...
[OK] Virtual environment created at: N:\RGD\OppNDA-TestEnv\venv

[3/5] Activating virtual environment...
[OK] Virtual environment activated

[4/5] Upgrading pip...
[OK] pip upgraded

[5/5] Installing requirements...
[OK] Requirements installed

Checking for outdated packages...

Would you like to update all packages? (y/N):

=====
Setup Complete
=====

To activate the virtual environment in the future, run:
..\.venv\Scripts\activate

To start OppNDA, run:
start.bat

Press any key to continue . . .
```

Figure 1: Running setup.bat on Windows

2.1.2 Linux/macOS

1. Clone and navigate to the repository:

```
git clone https://github.com/**/.git
cd OppNDA
```

2. Run the setup script:

```
bash scripts/setup.sh
```

3. Launch the application:

```
bash scripts/start.sh
```

2.2 Method 2: Manual Installation

For more control over the installation process:

```
# Create and activate virtual environment
python -m venv venv
source venv/bin/activate      # Linux/macOS
venv\Scripts\activate        # Windows

# Install dependencies
pip install -r requirements.txt

# Run the application
python OppNDA.py
```

2.3 Method 3: Docker Installation

For containerized deployment, OppNDA includes Docker support.

2.3.1 Using Docker Compose (Recommended)

```
# Build and run with Docker Compose
docker-compose up --build
```

2.3.2 Manual Docker Build

```
# Build the Docker image
docker build -t oppnda .

# Run the container
docker run -p 5001:5001 --name OppNDA oppnda
```

2.4 Accessing the Web Interface

After launching OppNDA, open your web browser and navigate to:

<http://localhost:5001/>

3 Scenario Configuration

The Settings page provides a comprehensive interface for configuring ONE simulator scenarios. This section covers both the creation of new configurations and the import of existing ones.

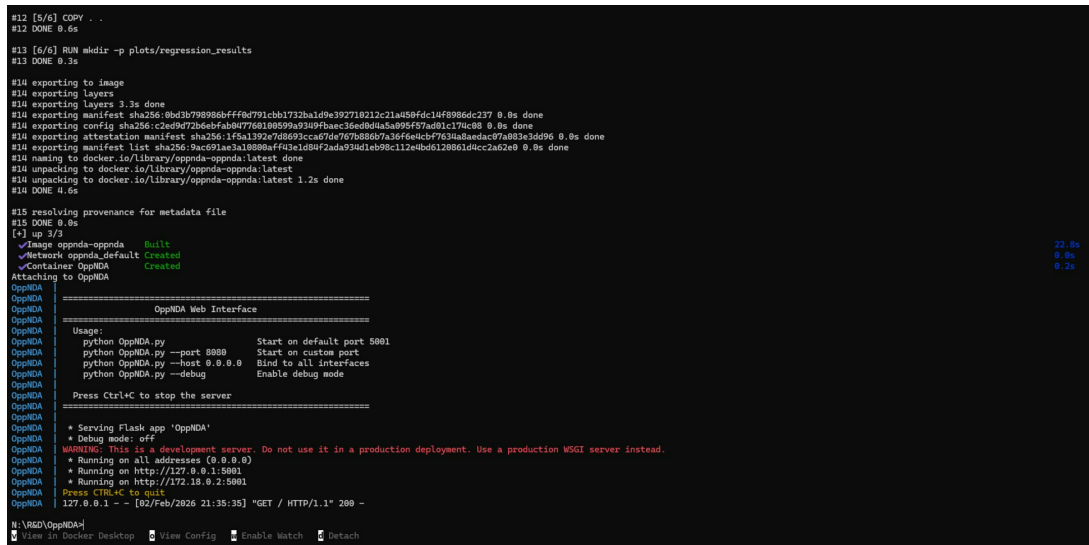


Figure 2: OppNDA running in a Docker container

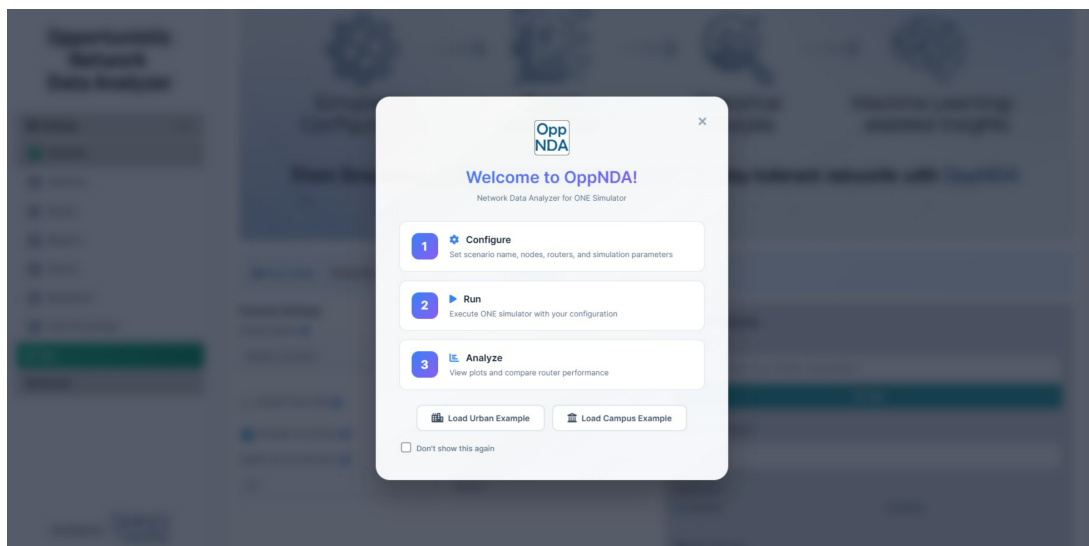


Figure 3: OppNDA web interface home page

3.1 Importing Existing Configuration Files

OppNDA supports importing existing ONE simulator configuration files (.txt format) directly into the GUI.

1. Click the **Import Config** button in the toolbar
2. Select your existing ONE configuration file
3. The GUI will parse and populate all settings automatically
4. Review and modify imported settings as needed

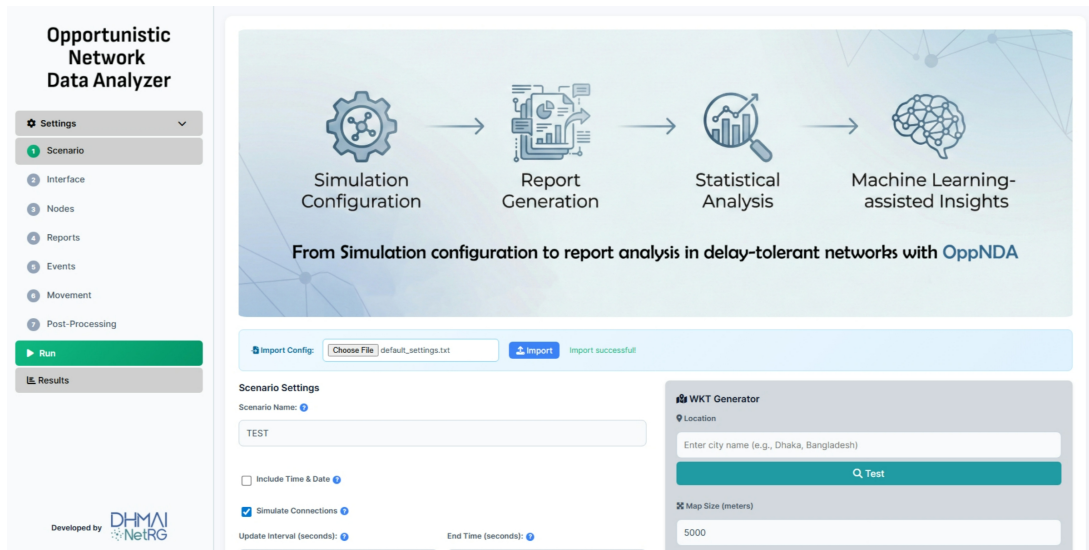


Figure 4: Importing an existing ONE configuration file

Note

When importing, OppNDA automatically recognizes standard ONE parameters and maps them to the corresponding GUI fields. Unknown parameters are preserved and can be viewed in the advanced settings section.

3.2 Scenario Settings

The basic scenario settings define the simulation environment:

1. **Scenario Name:** Enter a descriptive name for your simulation
2. **Simulation Time:** Set the duration in seconds
3. **Update Interval:** Configure the simulation tick rate
4. **World Size:** Define the simulation area dimensions (X, Y)

3.3 Network Interfaces

Configure the communication interfaces available to nodes:

1. Click **Add Interface** to create a new interface

2. Select the interface type (e.g., SimpleBroadcastInterface)
3. Configure parameters:
 - Transmission speed
 - Transmission range

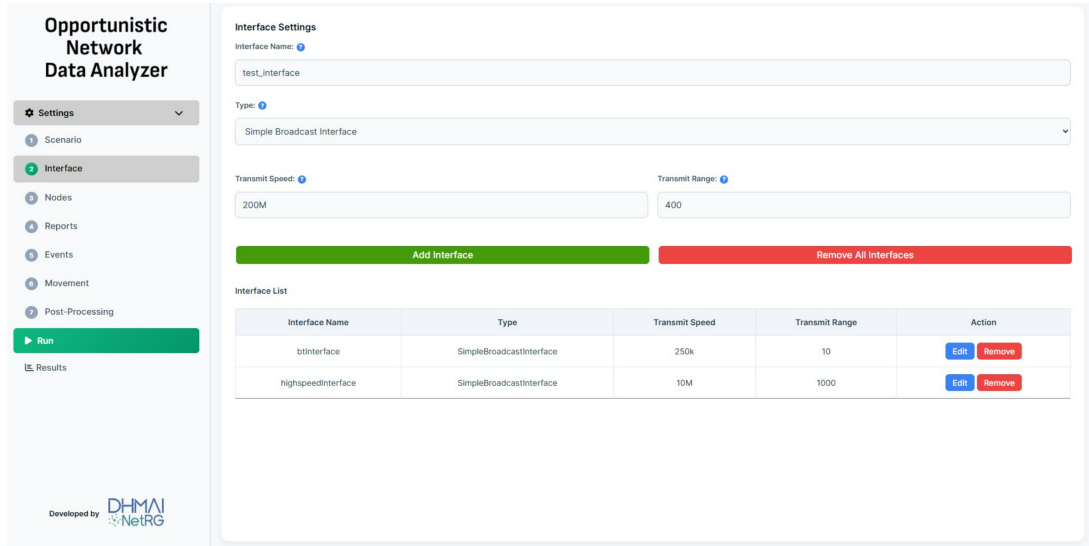


Figure 5: Network interface configuration

3.4 Host Groups

Define different categories of mobile nodes:

1. Click **Add Group** to create a new host group
2. Configure group parameters:
 - Group ID and number of hosts
 - Movement model (RandomWaypoint, ShortestPathMapBasedMovement, etc.)
 - Buffer size
 - Message TTL (Time-To-Live)
 - Router type (Epidemic, SprayAndWait, PProPHET, etc.)

3.5 Message Events

Configure message generation patterns:

1. Click **Add Event** to create a new event generator
2. Set event parameters:
 - Event class (MessageEventGenerator)
 - Message creation interval (min, max)
 - Message size range
 - Source and destination host ranges
 - Event timing (start, end)

Opportunistic
Network
Data Analyzer

Settings

Scenario

Interface

Nodes

Reports

Events

Movement

Post-Processing

Run

Results

Developed by DHMAI
NetRG

Energy Model Settings

☒ Enable Energy Model

Initial Energy (units):
100000

Scan Energy (units/scan):
10

Transmit Energy (units/byte):
20

Base Energy (units/sec):
5

Energy consumption: base + scan per discovery + transmit per byte sent

Custom Groups

Group ID:
smarLEV

Number of Hosts:
4

Movement Model:
Map Based Movement

Route Type:
Type 2

Wait Time Min (seconds):
0

Wait Time Max (seconds):
100

Speed Min (m/s):
10

Speed Max (m/s):
30

Figure 6: Host group configuration panel

Opportunistic
Network
Data Analyzer

Settings

Scenario

Interface

Nodes

Reports

Events

Movement

Post-Processing

Run

Results

Developed by DHMAI
NetRG

Event Settings

Configure message generation events for the simulation. Events define how and when messages are created between nodes.

Add New Event

Event Class:
MessageEventGenerator

Creation Interval (seconds):
25 to 35

Message Size:
500k to 1M
Use k for KB, M for MB (e.g., 500k, 1M)

Source/Destination Hosts:
0 to 126
Host address range (typically 0 to total hosts - 1)

Message ID Prefix:
Test

+ Add Event

Remove All Events

Configured Events

Event Class	Creation Interval (s)	Message Size	Hosts	Prefix	Action
MessageEventGenerator	25, 35	500k, 1M	0, 220	M	Delete

Figure 7: Message event generator configuration

3.6 Report Selection

Choose which reports the ONE simulator should generate:

1. Browse the available report types
2. Select desired reports (e.g., MessageStatsReport, DeliveredMessagesReport)
3. Configure report-specific parameters if needed
4. Set the report output directory

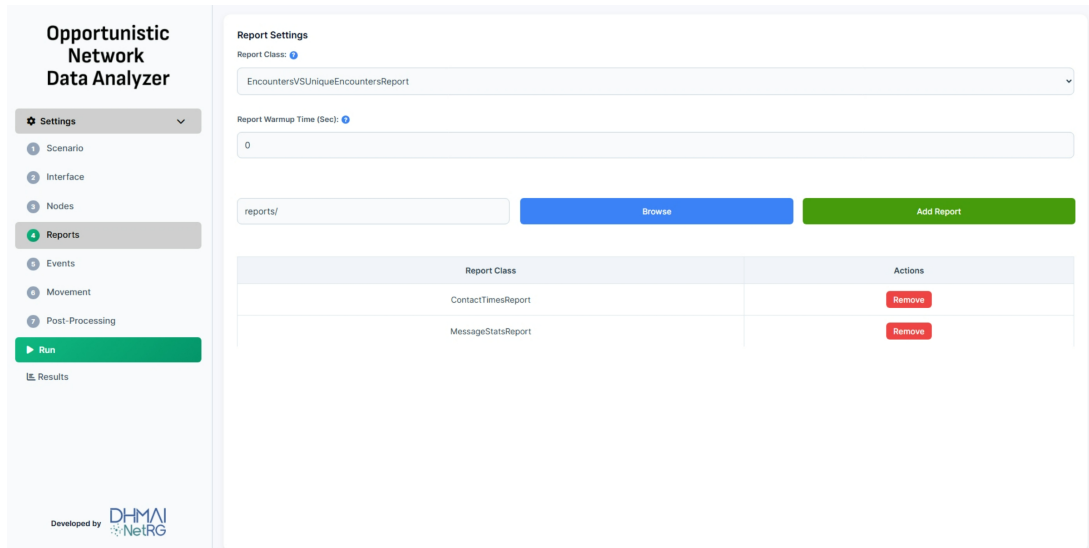


Figure 8: Report type selection panel

3.7 Saving Configuration

After configuring all parameters:

1. Click **Save Config** to save the current configuration
2. The configuration is saved as a `.txt` file compatible with ONE
3. OppNDA also maintains JSON backups of all settings

4 Running the ONE Simulator

OppNDA provides integrated execution of the ONE simulator with real-time output monitoring.

4.1 Starting a Simulation

1. Ensure your configuration is saved
2. Click the **Run ONE** button
3. OppNDA will:
 - Save the current configuration
 - Launch the ONE simulator with appropriate parameters
 - Display real-time console output
 - Automatically trigger post-processing upon completion

4.2 Monitoring Progress

The console panel displays real-time output from the simulator:

- Simulation progress percentage
- Current simulation time
- Messages created and delivered
- Any warnings or errors

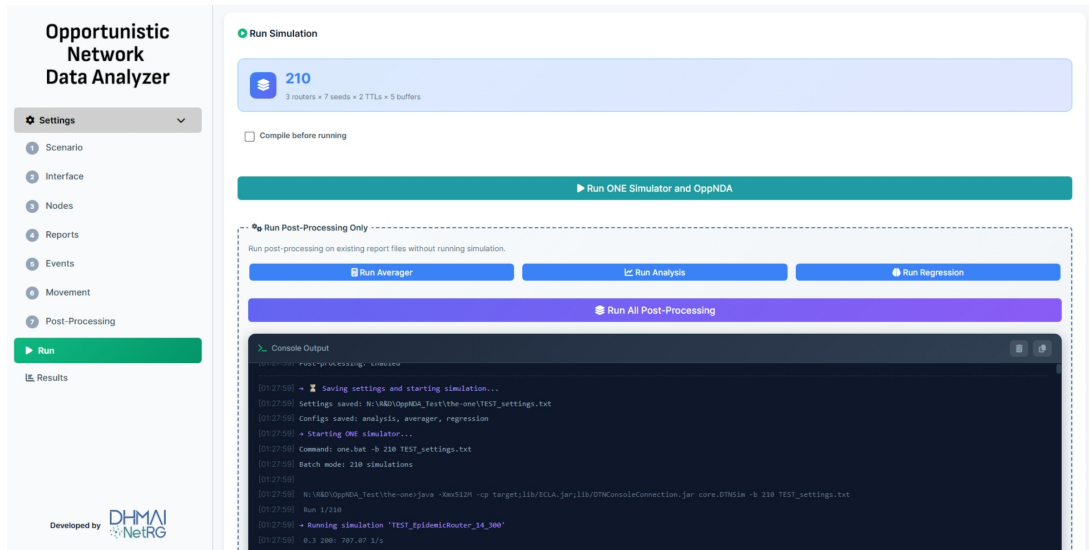


Figure 9: Real-time console output during simulation

Tip

For batch simulations with multiple parameter combinations, you can configure seed ranges and parameter variations. OppNDA will execute simulations sequentially and aggregate results.

4.3 Terminating a Simulation

If needed, you can stop a running simulation:

1. Click the **Terminate** button next to the Run ONE button
2. Confirm the termination
3. Any partial results will be preserved

5 Post-Processing: Report Averaging

The averaging module aggregates results across multiple simulation runs (seeds) to produce statistically meaningful data.

5.1 Understanding Filename Patterns

OppNDA uses configurable filename patterns to group and average reports. An example of report filename follows this structure:

```
RouterType_TTL_BufferSize_Seed_ReportType.txt
```

For example: Epidemic_300_5M_1_MessageStatsReport.txt

5.2 Configuring the Average class

1. Navigate to the **Post-Processing** section
2. Set the **Input Directory** containing raw report files
3. Configure the **Filename Pattern** using the drag-and-drop pattern builder
4. Set **Grouping Parameters** to define which parameters to group by (e.g., Router, TTL, Buffer)
5. Set the **Seed Position** to identify which part of the filename represents the seed number

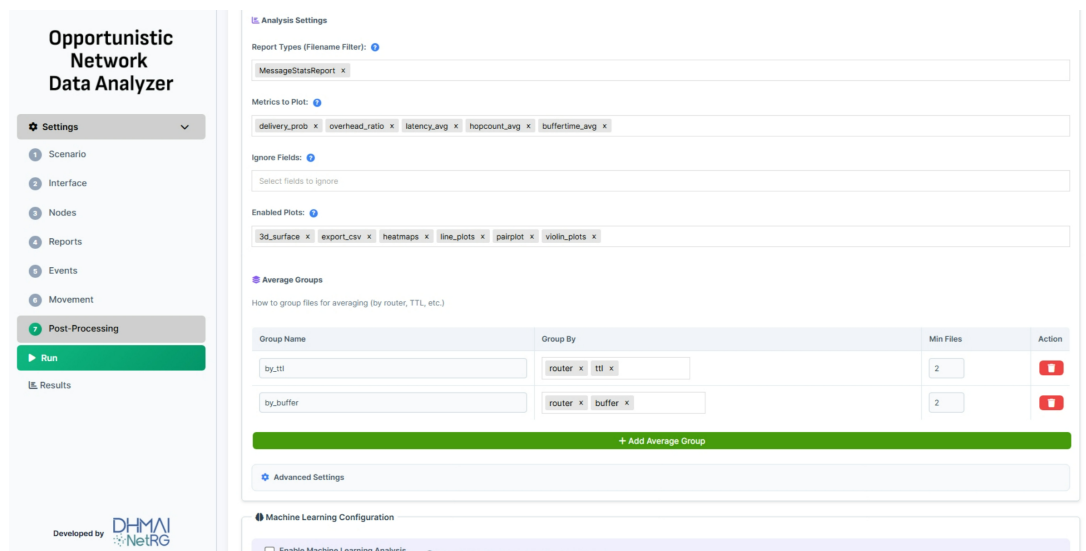


Figure 10: *Averager* configuration panel

5.3 Pattern Builder

The interactive pattern builder helps define filename patterns:

1. Drag components (Router, TTL, Buffer, Seed, etc.) into position
2. Set delimiters between components (underscore, hyphen, etc.)
3. Preview how patterns match your actual files

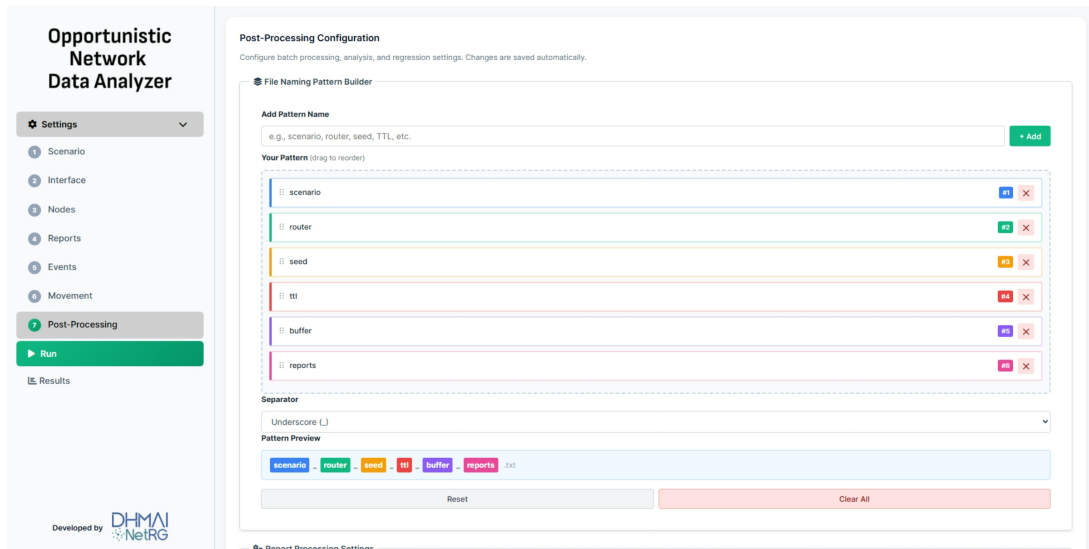


Figure 11: Interactive pattern builder

5.4 Running the 'Averager'

1. Click **Run Averager**
2. Monitor progress in the console panel
3. Averaged files are saved with **averaged_** prefix in the output directory

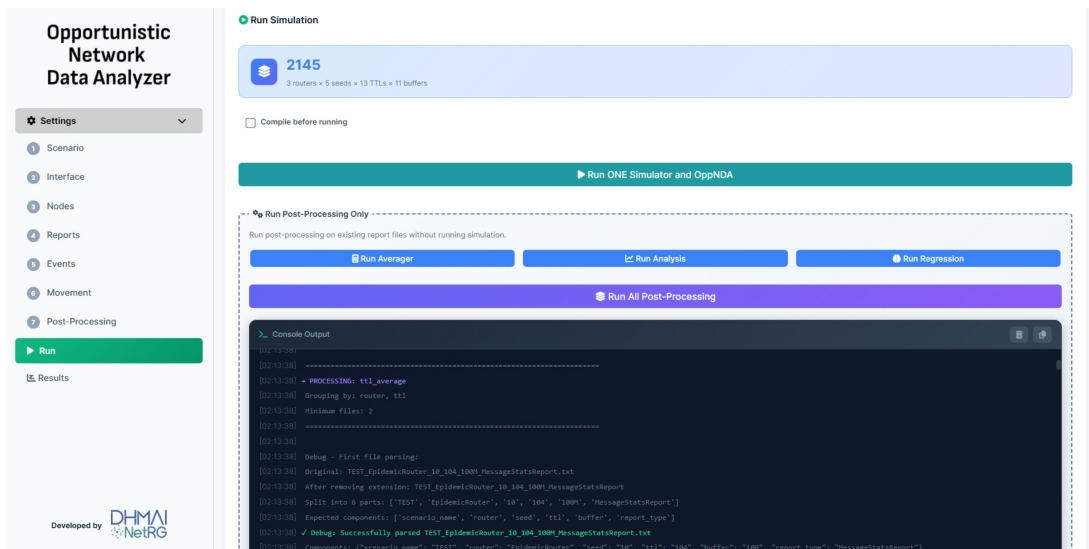


Figure 12: *Averager* console output showing grouped files

5.5 Understanding Averaged Output

The averaged output files contain:

- Mean values across all seeds
- Standard deviation for each metric
- Count of samples averaged

6 Post-Processing: Visualization and Analysis

The analysis module generates publication-ready visualizations from averaged data.

6.1 Configuring Analysis

1. Select the **Input Directory** containing averaged reports
2. Choose **Report Types** to analyze
3. Configure axis parameters (X, Y, Z variables)
4. Set desired **Plot Types**

Group Name	Group By	Min Files	Action
by_ttl	router x ttl x	2	
by_buffer	router x buffer x	2	

Figure 13: Analysis configuration panel

6.2 Available Plot Types

OppNDA supports multiple visualization types:

- **3D Surface Plots:** Visualize relationships between three variables
- **Line Plots:** Show trends across parameter values
- **Heatmaps:** Display intensity matrices
- **Violin Plots:** Show data distribution
- **Pair Plots:** Explore correlations between metrics

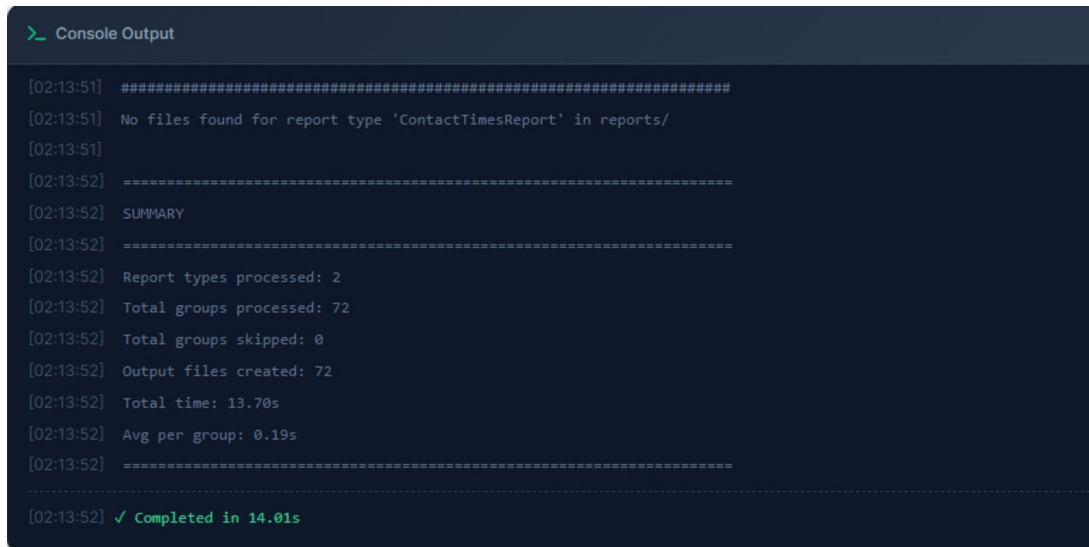
6.3 Plot Settings

Customize plot appearance:

1. **Figure Size:** Set dimensions in inches
2. **Font Sizes:** Configure title, label, and tick fonts
3. **Color Scheme:** Choose from available color palettes
4. **DPI:** Set resolution for saved images

6.4 Running Analysis

1. Click **Run Analysis**
2. Monitor progress as plots are generated
3. Generated plots are saved to the **plots/** directory



```
> Console Output

[02:13:51] #####
[02:13:51] No files found for report type 'ContactTimesReport' in reports/
[02:13:51] #####
[02:13:52] #####
[02:13:52] SUMMARY
[02:13:52] #####
[02:13:52] Report types processed: 2
[02:13:52] Total groups processed: 72
[02:13:52] Total groups skipped: 0
[02:13:52] Output files created: 72
[02:13:52] Total time: 13.70s
[02:13:52] Avg per group: 0.19s
[02:13:52] #####
[02:13:52] ✓ Completed in 14.01s
```

Figure 14: Analysis progress with real-time logging

6.5 Viewing Generated Plots

After analysis completes:

- View plots directly in the web interface
- Access high-resolution versions in the **plots/** directory
- Download plots in PNG or PDF format

7 Regression Analysis

The regression module trains machine learning models to understand and predict network performance metrics.

7.1 Preparing Input Data

Regression analysis uses CSV files generated during the analysis phase:

1. Ensure analysis has been run to generate CSV output
2. The CSV files contain structured data suitable for ML training

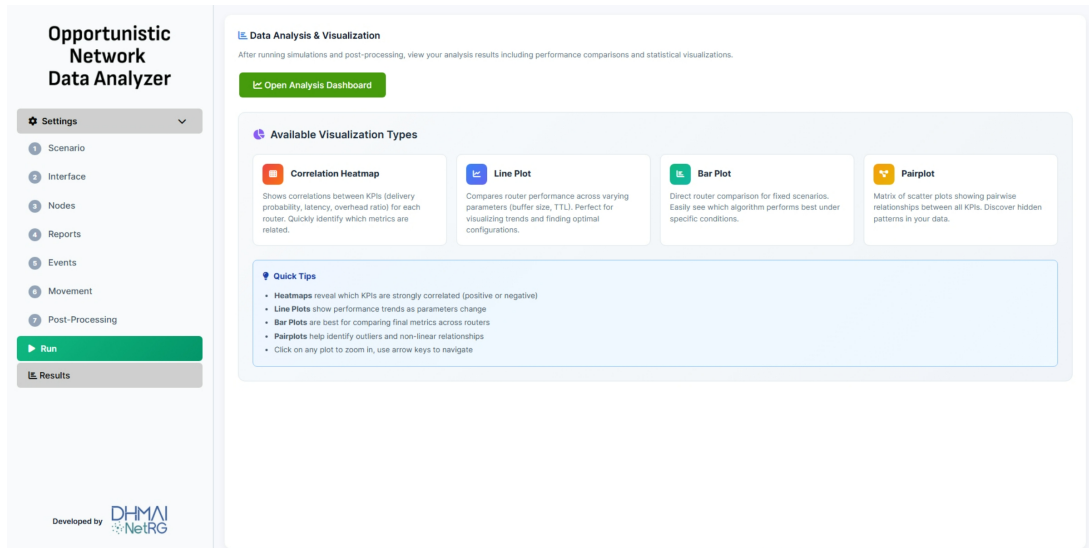


Figure 15: View results generated from simulation reports

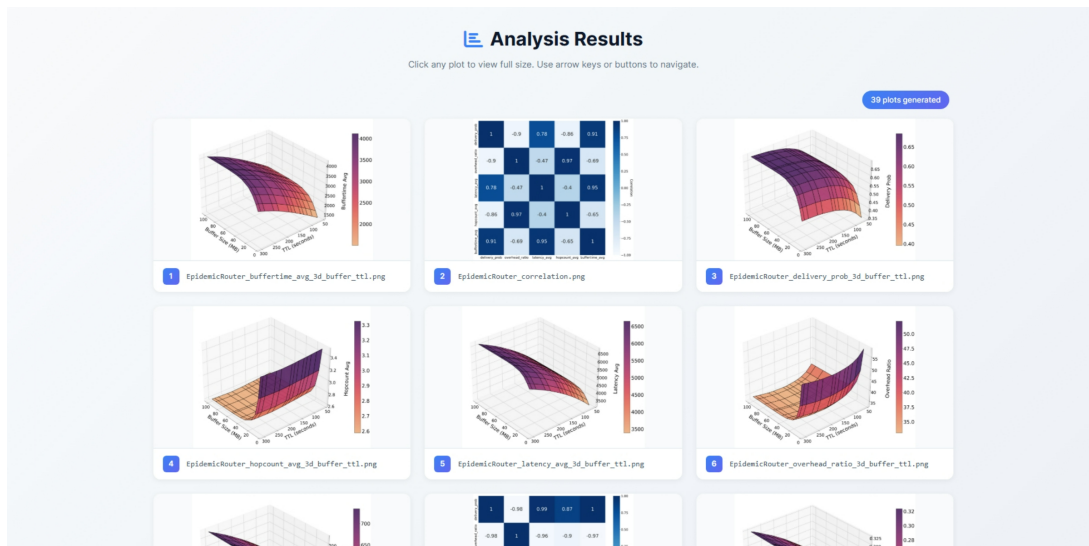


Figure 16: Visualizations of simulation data

7.2 Configuring Regression

1. Navigate to the **Regression** section
2. Select the **Input CSV File**
3. Choose **Target Variable(s)** to predict (e.g., delivery_ratio, latency)
4. Select **Predictor Variables** (e.g., TTL, buffer_size, num_hosts)
5. Choose **ML Models** to train

The screenshot shows the 'Regression configuration panel' within the 'OppNDA' application. On the left is a sidebar with a 'Settings' dropdown and a list of menu items: Scenario, Interface, Nodes, Reports, Events, Movement, Post-Processing, Run, and Results. The 'Run' button is highlighted in green. The main panel is titled 'Machine Learning Configuration' and contains several sections: 'Advanced Settings' (with a toggle for 'Enable Machine Learning Analysis'), 'Results Folder' (set to 'regression_results'), 'Target Variable(s)' (set to 'delivery_prob'), 'Predictor Variables' (a list of selected variables including BufferSize, TTL, overhead_ratio, hopcount_avg, latency_avg, and buftime_avg), 'Exclude Variables' (set to 'seed' and 'run_id'), 'Train Size (0-1)' (set to 0.75), 'Random State' (set to 5), 'Enabled Models' (a list of selected models including Decision Tree, Gradient Boosting, KNN, Linear Regression, and Random Forest), and checkboxes for 'Normalize Features', 'Polynomial Features', and 'Cross Validation'. The bottom of the panel has an 'Advanced Settings' link.

Figure 17: Regression configuration panel

The screenshot shows the 'Advanced regression configuration' panel. It includes a 'Model Tuning' section with a note about fine-tuning parameters. The 'Polynomial Features' section has a 'Degree' set to 2 and checkboxes for 'Interaction Only' and 'Include Bias'. The 'Cross Validation' section has a 'Number of Folds' set to 5. The 'Model Parameters' section contains hyperparameters for different models: 'KNN Neighbors' (6), 'Decision Tree Max Depth' (Empty = unlimited), 'Random Forest Trees' (100), 'GB Boosting Stages' (100), 'GB Learning Rate' (0.1), and 'GB Max Depth' (3). The bottom of the panel has an 'Advanced Settings' link.

Figure 18: Advanced regression configuration

7.3 Running Regression

1. Click **Run Regression**
2. Monitor training progress

3. View model performance metrics (R^2 , RMSE, MAE)

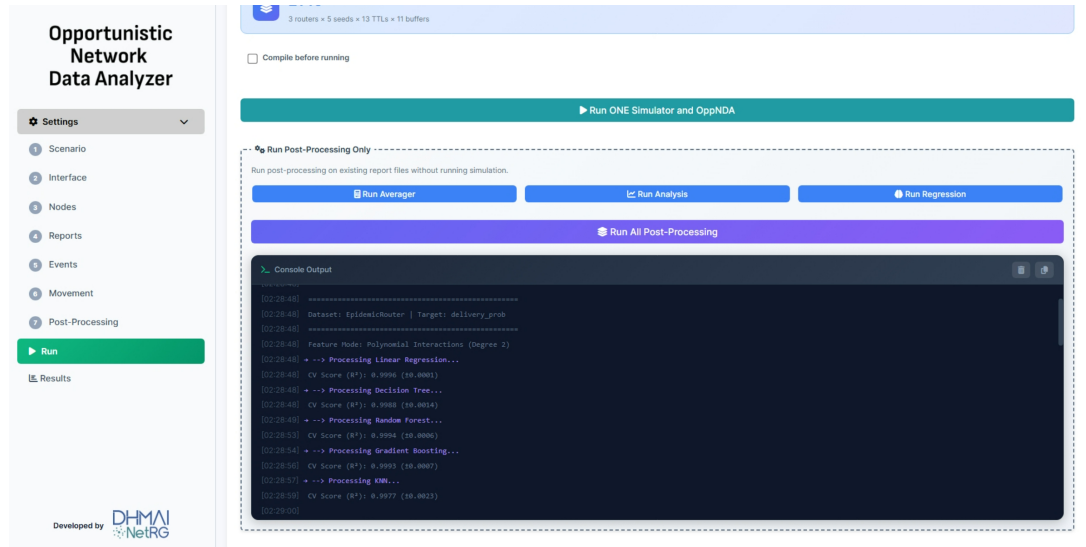


Figure 19: Regression training progress

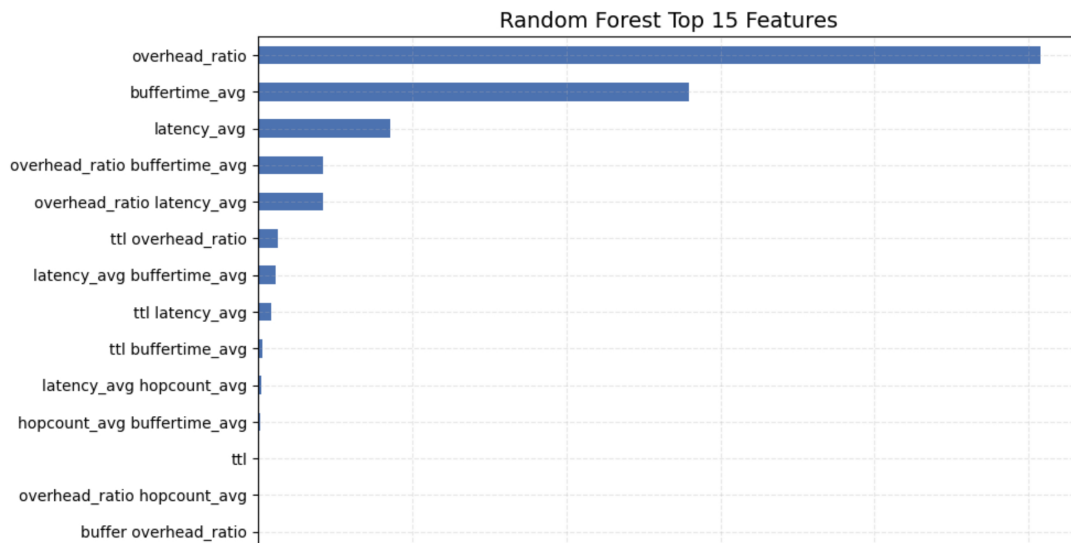


Figure 20: Feature importance of spray and wait router using random forest

Tip

Use feature importance to identify which simulation parameters most strongly affect network performance metrics. This can inform future parameter selection for simulations.